

AdControl in XAML and .NET

Article • 03/16/2023

⚠ Warning

As of June 1, 2020, the Microsoft Ad Monetization platform for Windows UWP apps will be shut down. [Learn more](#) [↗]

This walkthrough shows how to use the [AdControl](#) class to display banner ads in a Universal Windows Platform (UWP) XAML app for Windows 10 or Windows 11 that is implemented using C#.

📌 Note

The Microsoft Advertising SDK also supports XAML apps that are implemented using C++. For a complete sample project, see the [advertising samples on GitHub](#) [↗].

Prerequisites

- Install the [Microsoft Advertising SDK](#) [↗] with Visual Studio 2015 or a later release of Visual Studio. For installation instructions, see [this article](#).

Integrate a banner ad into your app

1. In Visual Studio, open your project or create a new project.

📌 Note

If you're using an existing project, open the Package.appxmanifest file in your project and ensure that the **Internet (Client)** capability is selected. Your app needs this capability to receive test ads and live ads.

2. If your project targets **Any CPU**, update your project to use an architecture-specific build output (for example, **x86**). If your project targets **Any CPU**, you will not be able to successfully add a reference to the Microsoft advertising library in the following steps. For more information, see [Reference errors caused by targeting Any CPU in your project](#).

3. Add a reference to the Microsoft Advertising SDK in your project:
 - a. From the **Solution Explorer** window, right click **References**, and select **Add Reference...**
 - b. In **Reference Manager**, expand **Universal Windows**, click **Extensions**, and then select the check box next to **Microsoft Advertising SDK for XAML** (Version 10.0).
 - c. In **Reference Manager**, click OK.
4. Modify the XAML for the page where you are embedding advertising to include the **Microsoft.Advertising.WinRT.UI** namespace. For example, in the default sample app generated by Visual Studio (named, in this app, MyAdFundedWindows10AppXAML), the XAML page is **MainPage.XAML**.

The **Page** section of the MainPage.xaml file generated by Visual Studio has the following code.

XML

```
<Page
  x:Class="MyAdFundedWindows10AppXAML.MainPage"
  xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
  xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
  xmlns:local="using:MyAdFundedWindows10AppXAML"
  xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
  xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
  mc:Ignorable="d">
  <Grid Background="{ThemeResource
ApplicationPageBackgroundThemeBrush}">
  </Grid>
</Page>
```

Add the namespace reference **Microsoft.Advertising.WinRT.UI** so the **Page** section of the MainPage.xaml file has the following code.

XML

```
<Page
  x:Class="MyAdFundedWindows10AppXAML.MainPage"
  xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
  xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
  xmlns:local="using:MyAdFundedWindows10AppXAML"
  xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
  xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
  xmlns:UI="using:Microsoft.Advertising.WinRT.UI"
  mc:Ignorable="d">
  <Grid Background="{ThemeResource
```

```
ApplicationPageBackgroundThemeBrush}">
    </Grid>
</Page>
```

5. In the **Grid** tag, add the code for the **AdControl**. Assign the [AdUnitId](#) and [ApplicationId](#) properties to the [test ad unit values](#). Also adjust the **Height** and **Width** of the control so it is one of the [supported ad sizes for banner ads](#).

ⓘ Note

Every **AdControl** has a corresponding *ad unit* that is used by our services to serve ads to the control, and every ad unit consists of an *ad unit ID* and *application ID*. In these steps, you assign test ad unit ID and application ID values to your control. These test values can only be used in a test version of your app. Before you publish your app to the Store, you must [replace these test values with live values](#) from Partner Center.

The complete **Grid** tag looks like this code.

XML

```
<Grid Background="{StaticResource
ApplicationPageBackgroundThemeBrush}">
    <UI:AdControl ApplicationId="00001111-aaaa-2222-bbbb-3333cccc4444"
        AdUnitId="test"
        HorizontalAlignment="Left"
        Height="250"
        VerticalAlignment="Top"
        Width="300"/>
</Grid>
```

The complete code for the MainPage.xaml file should look like this.

XML

```
<Page
    x:Class="MyAdFundedWindows10AppXAML.MainPage"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:local="using:MyAdFundedWindows10AppXAML"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006"
    xmlns:UI="using:Microsoft.Advertising.WinRT.UI"
    mc:Ignorable="d">
    <Grid Background="{StaticResource
ApplicationPageBackgroundThemeBrush}">
```

```
<UI:AdControl ApplicationId="00001111-aaaa-2222-bbbb-  
3333cccc4444"  
    AdUnitId="test"  
    HorizontalAlignment="Left"  
    Height="250"  
    VerticalAlignment="Top"  
    Width="300"/>  
  
</Grid>  
</Page>
```

6. Compile and run the app to see it with an ad.

Release your app with live ads

1. Make sure your use of banner ads in your app follows our [guidelines for banner ads](#).
2. In Partner Center, go to the [In-app ads](#) page and [create an ad unit](#). For the ad unit type, specify **Banner**. Make note of both the ad unit ID and the application ID.

ⓘ Note

The application ID values for test ad units and live UWP ad units have different formats. Test application ID values are GUIDs. When you create a live UWP ad unit in Partner Center, the application ID value for the ad unit always matches the Store ID for your app (an example Store ID value looks like 9NBLGGH4R315).

3. You can optionally enable ad mediation for the **AdControl** by configuring the settings in the [Mediation settings](#) section on the [In-app ads](#) page. Ad mediation enables you to maximize your ad revenue and app promotion capabilities by displaying ads from multiple ad networks, including ads from other paid ad networks such as Taboola and Smaato and ads for Microsoft app promotion campaigns.
4. In your code, replace the test ad unit values (**ApplicationId** and **AdUnitId**) with the live values you generated in Partner Center.
5. [Submit your app](#) to the Store using Partner Center.
6. Review your [advertising performance reports](#) in Partner Center.

Manage ad units for multiple ad controls in your app

You can use multiple **AdControl** objects in a single app (for example, each page in your app might host a different **AdControl** object). In this scenario, we recommend that you assign a different ad unit to each control. Using different ad units for each control enables you to separately [configure the mediation settings](#) and get discrete [reporting data](#) for each control. This also enables our services to better optimize the ads we serve to your app.

Important

You can use each ad unit in only one app. If you use an ad unit in more than one app, ads will not be served for that ad unit.

Related topics

- [Guidelines for banner ads](#)
- [Error handling in XAML/C# walkthrough.](#)
- [Advertising samples on GitHub](#) 
- [Set up ad units for your app](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

AdControl in HTML 5 and JavaScript

Article • 03/16/2023

⚠ Warning

As of June 1, 2020, the Microsoft Ad Monetization platform for Windows UWP apps will be shut down. [Learn more](#) [↗]

This walkthrough shows how to use the [AdControl](#) class to display banner ads in a Universal Windows Platform (UWP) JavaScript/HTML app for Windows 10 and Windows 11.

For a complete sample project that demonstrates how to add banner ads to a JavaScript/HTML app, see the [advertising samples on GitHub](#) [↗].

Prerequisites

- Install the [Microsoft Advertising SDK](#) [↗] with Visual Studio 2015 or a later release of Visual Studio. For installation instructions, see [this article](#).

ⓘ Note

If you have installed the Windows 10 SDK version 10.0.14393 (Anniversary Update) or a later version of the Windows SDK, you must also install the [WinJS](#) [↗] library. This library used to be included in previous versions of the Windows SDK for Windows 10, but starting with the Windows 10 SDK version 10.0.14393 (Anniversary Update) this library must be installed separately.

Integrate a banner ad into your app

1. In Visual Studio, open your project or create a new project.

ⓘ Note

If you're using an existing project, open the Package.appxmanifest file in your project and ensure that the **Internet (Client)** capability is selected. Your app needs this capability to receive test ads and live ads.

2. If your project targets **Any CPU**, update your project to use an architecture-specific build output (for example, **x86**). If your project targets **Any CPU**, you will not be able to successfully add a reference to the Microsoft advertising library in the following steps. For more information, see [Reference errors caused by targeting Any CPU in your project](#).
3. Add a reference to the Microsoft Advertising SDK in your project:
 - a. From the **Solution Explorer** window, right click **References**, and select **Add Reference...**
 - b. In **Reference Manager**, expand **Universal Windows**, click **Extensions**, and then select the check box next to **Microsoft Advertising SDK for JavaScript** (Version 10.0).
 - c. In **Reference Manager**, click OK.
4. Open the index.html file (or other html file as appropriate for your project).
5. In the **<head>** section, after the project's JavaScript references of default.css and main.js, add the reference to ad.js.

HTML

```
<!-- Advertising required references -->  
<script src="//Microsoft.Advertising.Javascript/ad.js"></script>
```

ⓘ Note

This line must be placed in the **<head>** section after the include of main.js; otherwise, you will encounter an error when you build your project.

6. Modify the **<body>** section in the default.html file (or other html file as appropriate for your project) to include the **div** for the **AdControl**. Assign the **applicationId** and **adUnitId** properties of the **AdControl** to the [test ad unit values](#). Also adjust the **height** and **width** so the control is one of the [supported ad sizes for banner ads](#).

ⓘ Note

Every **AdControl** has a corresponding *ad unit* that is used by our services to serve ads to the control, and every ad unit consists of an *ad unit ID* and *application ID*. In these steps, you assign test ad unit ID and application ID values to your control. These test values can only be used in a test version of

your app. Before you publish your app to the Store, you must replace these test values with live values from Partner Center.

HTML

```
<div id="myAd" style="position: absolute; top: 50px; left: 0px; width:
300px; height: 250px; z-index: 1"
    data-win-control="MicrosoftNSJS.Advertising.AdControl"
    data-win-options="{applicationId: '00001111-aaaa-2222-bbbb-
3333cccc4444', adUnitId: 'test'}">
</div>
```

7. Compile and run the app to see it with an ad.

The following example shows the complete index.html for a simple app.

HTML

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title>AdControlExampleApp</title>
    <!-- WinJS references -->
    <link href="lib/winjs-4.0.1/css/ui-light.css" rel="stylesheet" />
    <script src="lib/winjs-4.0.1/js/base.js"></script>
    <script src="lib/winjs-4.0.1/js/ui.js"></script>
    <!-- AdControlExampleApp references -->
    <link href="css/default.css" rel="stylesheet" />
    <script src="js/main.js"></script>
    <!-- Required reference for AdControl -->
    <script src="//Microsoft.Advertising.JavaScript/ad.js"></script>
</head>
<body>
    <div id="myAd" style="position: absolute; top: 50px; left: 0px; width:
300px; height: 250px; z-index: 1"
        data-win-control="MicrosoftNSJS.Advertising.AdControl"
        data-win-options="{applicationId: '00001111-aaaa-2222-bbbb-
3333cccc4444', adUnitId: 'test'}">
        </div>
    <p>Content goes here</p>
</body>
</html>
```

Create an AdControl programmatically in JavaScript

The previous steps show how to declare an **AdControl** in your HTML markup. Alternatively, you can programmatically create an **AdControl** using JavaScript. This

example assumes that you are using an existing **div** in your HTML with the ID **myAd**.

JavaScript

```
var adDiv = document.getElementById("myAd");
var myAdControl = new MicrosoftNSJS.Advertising.AdControl(adDiv,
{
    applicationId: "3f83fe91-d6be-434d-a0ae-7351c5a997f1",
    adUnitId: "test",
});

myAdControl.isAutoRefreshEnabled = false;
myAdControl.onErrorOccurred = myAdError;
myAdControl.onAdRefreshed = myAdRefreshed;
myAdControl.onEngagedChanged = myAdEngagedChanged;
```

This example assumes that you have already declared event handler methods named **myAdError**, **myAdRefreshed**, and **myAdEngagedChanged**.

If you use this code and do not see ads, you can try inserting an attribute of **position:relative** in the **div** that contains the **AdControl**. This will override the default setting of the **IFrame**. Ads will be displayed correctly, unless they are not being shown due to the value of this attribute. Note that new ad units may not be available for up to 30 minutes.

ⓘ Note

The *applicationId* and *adUnitId* values shown in this example are test mode values. You must replace these values with live values from Partner Center before submitting your app for submission.

Release your app with live ads

1. Make sure your use of banner ads in your app follows our [guidelines for banner ads](#).
2. In Partner Center, go to the [In-app ads](#) page and [create an ad unit](#). For the ad unit type, specify **Banner**. Make note of both the ad unit ID and the application ID.

ⓘ Note

The application ID values for test ad units and live UWP ad units have different formats. Test application ID values are GUIDs. When you create a live

UWP ad unit in Partner Center, the application ID value for the ad unit always matches the Store ID for your app (an example Store ID value looks like 9NBLGGH4R315).

3. You can optionally enable ad mediation for the **AdControl** by configuring the settings in the [Mediation settings](#) section on the [In-app ads](#) page. Ad mediation enables you to maximize your ad revenue and app promotion capabilities by displaying ads from multiple ad networks, including ads from other paid ad networks such as Taboola and Smaato and ads for Microsoft app promotion campaigns.
4. In your code, replace the test ad unit values (**applicationId** and **adUnitId**) with the live values you generated in Partner Center.
5. [Submit your app](#) to the Store using Partner Center.
6. Review your [advertising performance reports](#) in Partner Center.

Manage ad units for multiple ad controls in your app

You can use multiple **AdControl** objects in a single app (for example, each page in your app might host a different **AdControl** object). In this scenario, we recommend that you assign a different ad unit to each control. Using different ad units for each control enables you to separately [configure the mediation settings](#) and get discrete [reporting data](#) for each control. This also enables our services to better optimize the ads we serve to your app.

Important

You can use each ad unit in only one app. If you use an ad unit in more than one app, ads will not be served for that ad unit.

Related topics

- [Guidelines for banner ads](#)
- [Advertising samples on GitHub](#) 
- [Set up ad units for your app](#)
- [Error handling in JavaScript walkthrough](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Supported banner ad sizes

Article • 10/20/2022

Warning

As of June 1, 2020, the Microsoft Ad Monetization platform for Windows UWP apps will be shut down. [Learn more](#) 

The following banner ad sizes are supported for Universal Windows Platform (UWP) apps. When you instantiate your **AdControl** object in your app, make sure you set the width and height properties to match one of these supported sizes.

Sizes are width x height.

- 160x600
- 300x50
- 300x250
- 300x600
- 320x50
- 640x100
- 728x90

Note

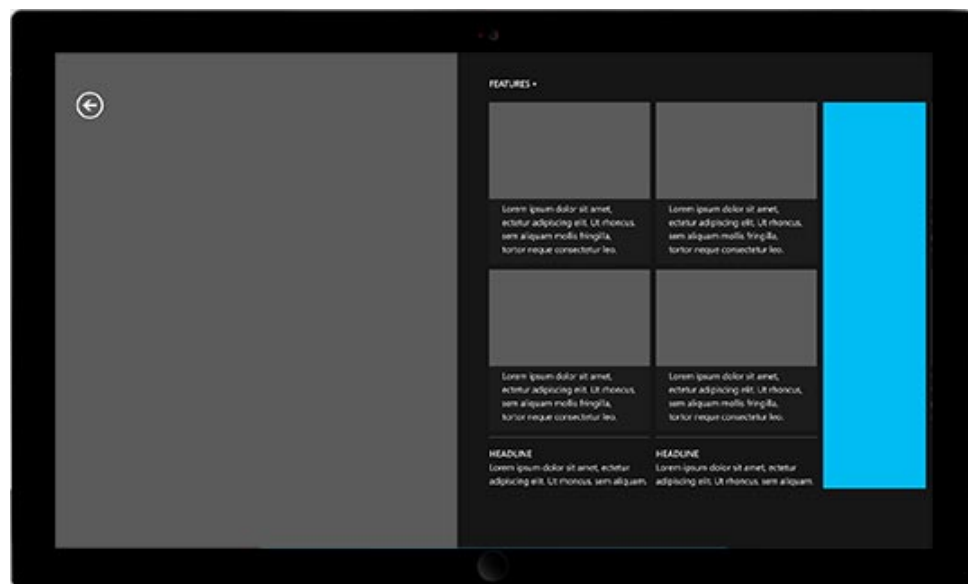
For the best experience, we recommend using the following sizes on Windows 10 Mobile devices: 300x50, 320x50, and 640x100.

The following table provides examples that illustrate each banner ad size.

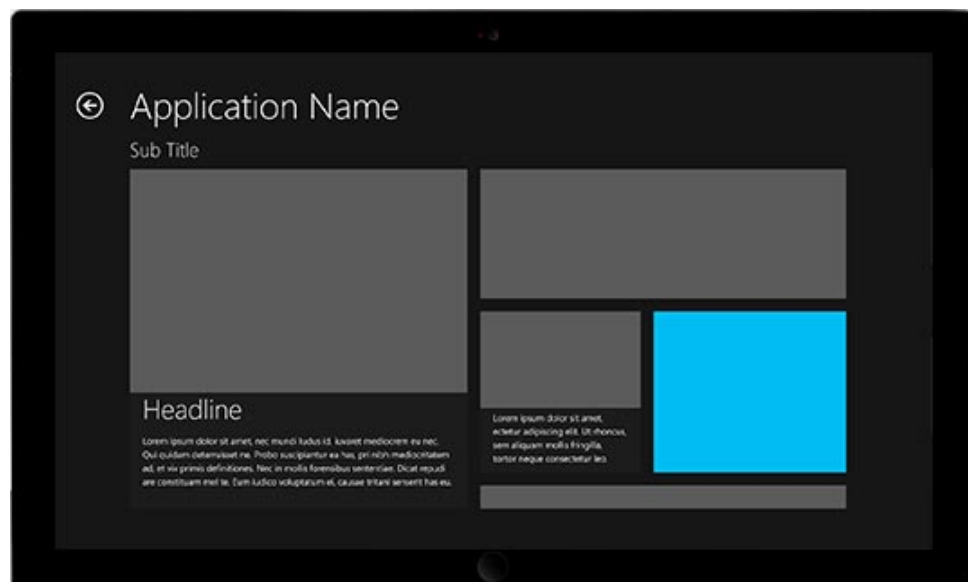
Ad size	Example
---------	---------

Ad size**Example**

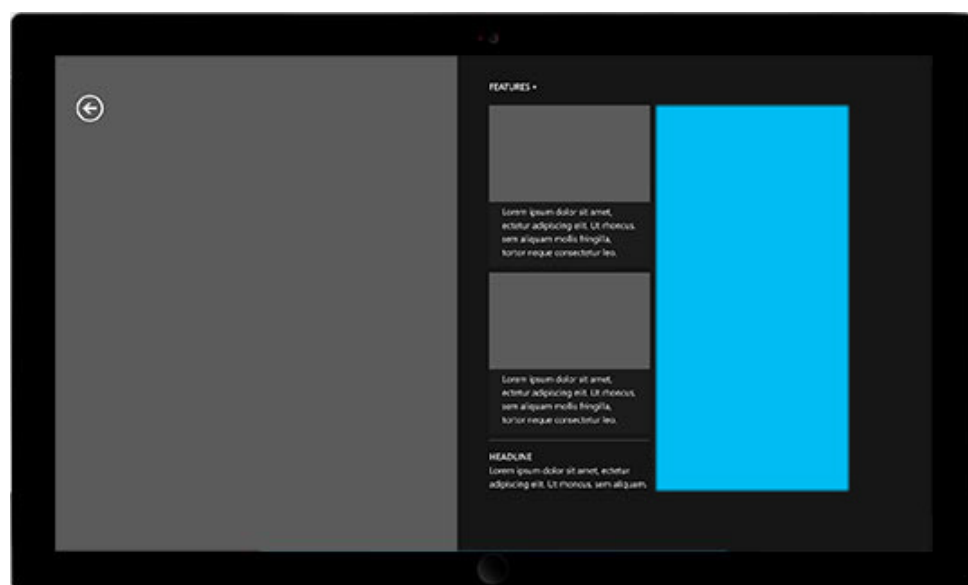
160x600



300x250

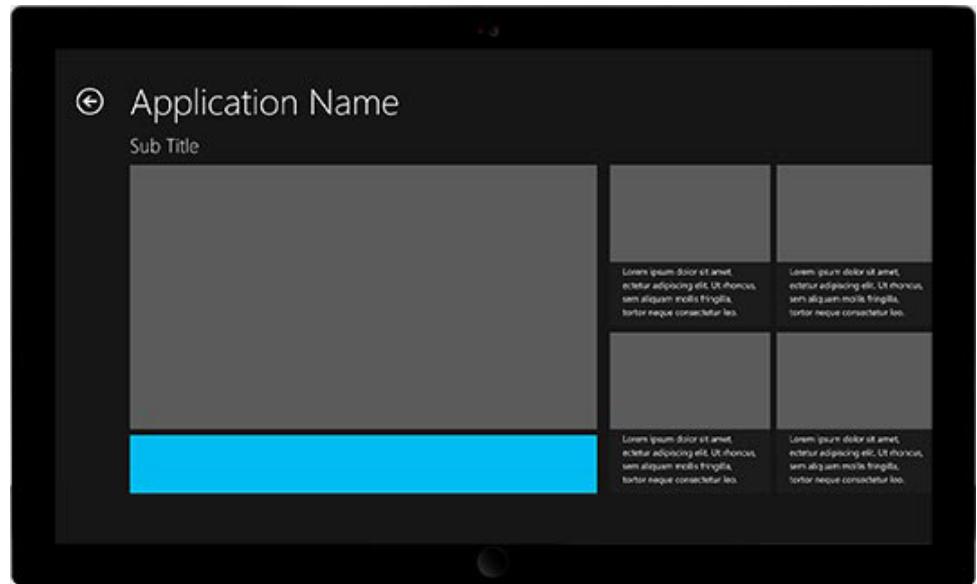


300x600

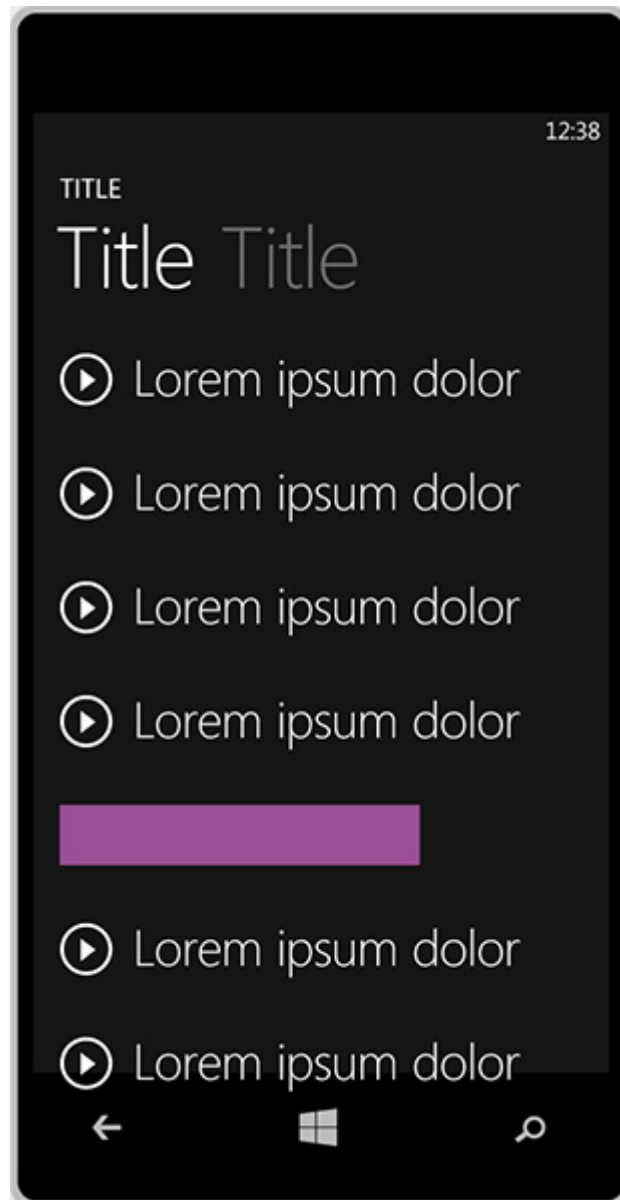


Ad size**Example**

728x90

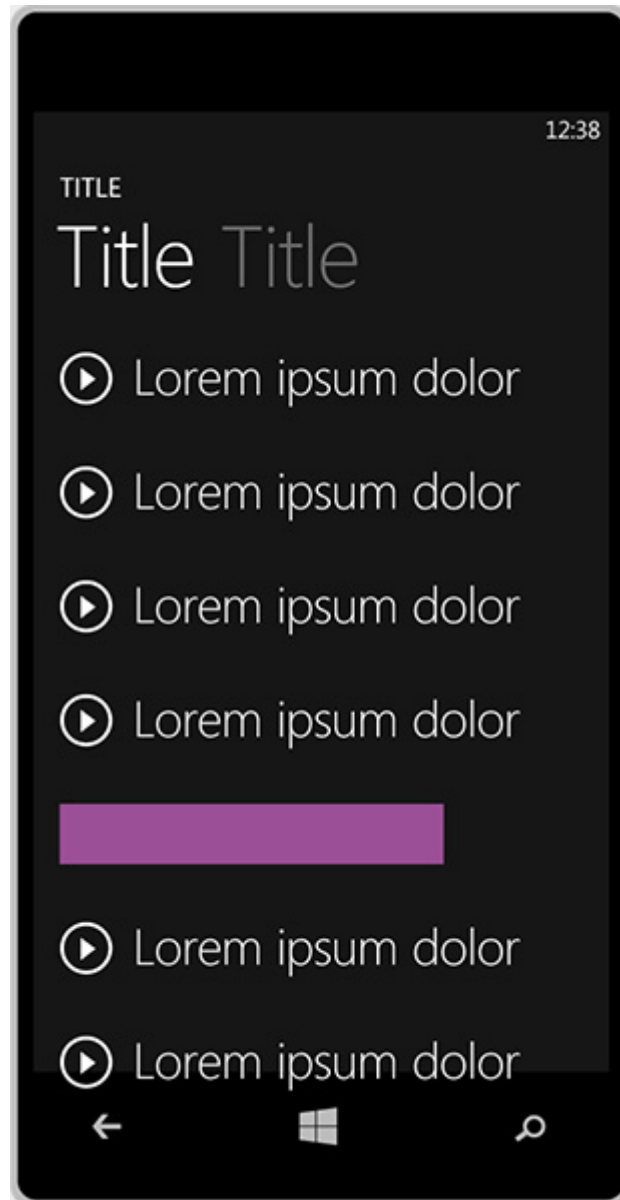


300x50

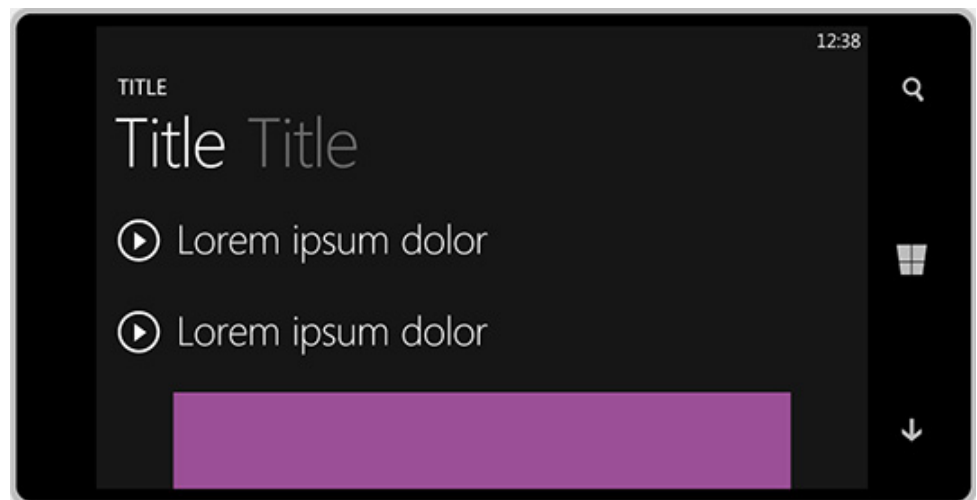


Ad size**Example**

320x50



640x100



Interstitial ads

Article • 03/16/2023

⚠ Warning

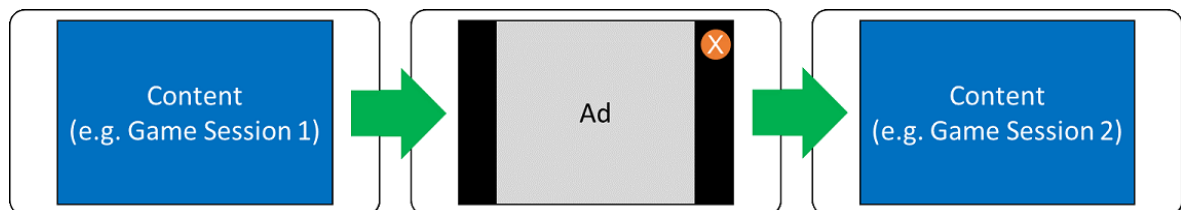
As of June 1, 2020, the Microsoft Ad Monetization platform for Windows UWP apps will be shut down. [Learn more](#) ↗

This walkthrough shows how to include interstitial ads in Universal Windows Platform (UWP) apps and games for Windows 10 and Windows 11. For complete sample projects that demonstrate how to add interstitial ads to JavaScript/HTML apps and XAML apps using C# and C++, see the [advertising samples on GitHub](#) ↗.

What are interstitial ads?

Unlike standard banner ads, which are confined to a portion of a UI in an app or game, interstitial ads are shown on the entire screen. Two basic forms are frequently used in games.

- With *Paywall* ads, the user must watch an ad at some regular interval. For example between game levels:



- With *Rewards Based* ads the user is explicitly seeking some benefit, such as a hint or extra time to complete the level, and initializes the ad through the app's user interface.

We provide two types of interstitial ads to use in your apps and games: **interstitial video ads** and **interstitial banner ads**.

📌 Note

The API for interstitial ads does not handle any user interface except at the time of video playback. Refer to the [interstitial best practices](#) for guidelines on what to do, and avoid, as you consider how to integrate interstitial ads in your app.

Prerequisites

- Install the [Microsoft Advertising SDK](#) with Visual Studio 2015 or a later release of Visual Studio. For installation instructions, see [this article](#).

Integrate an interstitial ad into your app

To show interstitial ads in your app, follow the instructions for project type:

- [XAML/.NET](#)
- [HTML/JavaScript](#)
- [C++ \(DirectX Interop\)](#)

XAML/.NET

This section provides C# examples, but Visual Basic and C++ are also supported for XAML/.NET projects. For a complete C# code example, see [Interstitial ad sample code in C#](#).

1. Open your project in Visual Studio.

ⓘ Note

If you're using an existing project, open the Package.appxmanifest file in your project and ensure that the **Internet (Client)** capability is selected. Your app needs this capability to receive test ads and live ads.

2. If your project targets **Any CPU**, update your project to use an architecture-specific build output (for example, **x86**). If your project targets **Any CPU**, you will not be able to successfully add a reference to the Microsoft advertising library in the following steps. For more information, see [Reference errors caused by targeting Any CPU in your project](#).
3. Add a reference to the Microsoft Advertising SDK in your project:
 - a. From the **Solution Explorer** window, right click **References**, and select **Add Reference...**
 - b. In **Reference Manager**, expand **Universal Windows**, click **Extensions**, and then select the check box next to **Microsoft Advertising SDK for XAML** (Version 10.0).
 - c. In **Reference Manager**, click OK.

4. In the appropriate code file in your app (for example, in MainPage.xaml.cs or a code file for some other page), add the following namespace reference.

C#

```
using Microsoft.Advertising.WinRT.UI;
```

5. In an appropriate location in your app (for example, in `MainPage` or some other page), declare an `InterstitialAd` object and several string fields that represent the application ID and ad unit ID for your interstitial ad. The following code example assigns the `myAppId` and `myAdUnitId` fields to the `test values` for interstitial ads.

ⓘ Note

Every **InterstitialAd** has a corresponding *ad unit* that is used by our services to serve ads to the control, and every ad unit consists of an *ad unit ID* and *application ID*. In these steps, you assign test ad unit ID and application ID values to your control. These test values can only be used in a test version of your app. Before you publish your app to the Store, you must **replace these test values with live values** from Partner Center.

C#

```
InterstitialAd myInterstitialAd = null;  
string myAppId = "d25517cb-12d4-4699-8bdc-52040c712cab";  
string myAdUnitId = "test";
```

6. In code that runs on startup (for example, in the constructor for the page), instantiate the **InterstitialAd** object and wire up event handlers for events of the object.

C#

```
myInterstitialAd = new InterstitialAd();  
myInterstitialAd.AdReady += MyInterstitialAd_AdReady;  
myInterstitialAd.ErrorOccurred += MyInterstitialAd_ErrorOccurred;  
myInterstitialAd.Completed += MyInterstitialAd_Completed;  
myInterstitialAd.Cancelled += MyInterstitialAd_Cancelled;
```

7. If you want to show an *interstitial video* ad: Approximately 30-60 seconds before you need the ad, use the `RequestAd` method to pre-fetch the ad. This allows

enough time to request and prepare the ad before it should be shown. Be sure to specify **AdType.Video** for the ad type.

C#

```
myInterstitialAd.RequestAd(AdType.Video, myAppId, myAdUnitId);
```

If you want to show an *interstitial banner* ad: Approximately 5-8 seconds before you need the ad, use the [RequestAd](#) method to pre-fetch the ad. This allows enough time to request and prepare the ad before it should be shown. Be sure to specify **AdType.Display** for the ad type.

C#

```
myInterstitialAd.RequestAd(AdType.Display, myAppId, myAdUnitId);
```

- At the point in your code where you want to show the interstitial video or interstitial banner ad, confirm that the **InterstitialAd** is ready to be shown and then show it by using the [Show](#) method.

C#

```
if (InterstitialAdState.Ready == myInterstitialAd.State)
{
    myInterstitialAd.Show();
}
```

- Define the event handlers for the **InterstitialAd** object.

C#

```
void MyInterstitialAd_AdReady(object sender, object e)
{
    // Your code goes here.
}

void MyInterstitialAd_ErrorOccurred(object sender, AdErrorEventArgs e)
{
    // Your code goes here.
}

void MyInterstitialAd_Completed(object sender, object e)
{
    // Your code goes here.
}

void MyInterstitialAd_Cancelled(object sender, object e)
```

```
{  
    // Your code goes here.  
}
```

10. Build and test your app to confirm it is showing test ads.

HTML/JavaScript

The following instructions assume you have created a Universal Windows project for JavaScript in Visual Studio and are targeting a specific CPU. For a complete code sample, see [Interstitial ad sample code in JavaScript](#).

1. Open your project in Visual Studio.
2. If your project targets **Any CPU**, update your project to use an architecture-specific build output (for example, **x86**). If your project targets **Any CPU**, you will not be able to successfully add a reference to the Microsoft advertising library in the following steps. For more information, see [Reference errors caused by targeting Any CPU in your project](#).
3. Add a reference to the Microsoft Advertising SDK in your project:
 - a. From the **Solution Explorer** window, right click **References**, and select **Add Reference...**
 - b. In **Reference Manager**, expand **Universal Windows**, click **Extensions**, and then select the check box next to **Microsoft Advertising SDK for JavaScript** (Version 10.0).
 - c. In **Reference Manager**, click OK.
4. In the **<head>** section of the HTML file in the project, after the project's JavaScript references of default.css and default.js, add the reference to ad.js.

HTML

```
<script src="//Microsoft.Advertising.Javascript/ad.js"></script>
```

5. In a .js file in your project, declare an [InterstitialAd](#) object and several fields that contain the application ID and ad unit ID for your interstitial ad. The following code example assigns the `applicationId` and `adUnitId` fields to the [test values](#) for interstitial ads.

ⓘ **Note**

Every **InterstitialAd** has a corresponding *ad unit* that is used by our services to serve ads to the control, and every ad unit consists of an *ad unit ID* and *application ID*. In these steps, you assign test ad unit ID and application ID values to your control. These test values can only be used in a test version of your app. Before you publish your app to the Store, you must **replace these test values with live values** from Partner Center.

JavaScript

```
var interstitialAd = null;
var applicationId = "d25517cb-12d4-4699-8bdc-52040c712cab";
var adUnitId = "test";
```

6. In code that runs on startup (for example, in the constructor for the page), instantiate the **InterstitialAd** object and wire up event handlers for the object.

JavaScript

```
interstitialAd = new MicrosoftNSJS.Advertising.InterstitialAd();
interstitialAd.onErrorOccurred = errorOccurredHandler;
interstitialAd.onAdReady = adReadyHandler;
interstitialAd.onCancelled = cancelledHandler;
interstitialAd.onCompleted = completedHandler;
```

7. If you want to show an *interstitial video* ad: Approximately 30-60 seconds before you need the ad, use the [RequestAd](#) method to pre-fetch the ad. This allows enough time to request and prepare the ad before it should be shown. Be sure to specify **InterstitialAdType.video** for the ad type.

JavaScript

```
if (interstitialAd) {

    interstitialAd.requestAd(MicrosoftNSJS.Advertising.InterstitialAdType.v
        ideo, applicationId, adUnitId);
}
```

If you want to show an *interstitial banner* ad: Approximately 5-8 seconds before you need the ad, use the [RequestAd](#) method to pre-fetch the ad. This allows enough time to request and prepare the ad before it should be shown. Be sure to specify **InterstitialAdType.display** for the ad type.

JavaScript

```
if (interstitialAd) {

    interstitialAd.requestAd(MicrosoftNSJS.Advertising.InterstitialAdType.display, applicationId, adUnitId);
}
```

- At the point in your code where you want to show the ad, confirm that the **InterstitialAd** is ready to be shown and then show it by using the [Show](#) method.

JavaScript

```
if (interstitialAd && interstitialAd.state ===
    MicrosoftNSJS.Advertising.InterstitialAdState.ready) {
    interstitialAd.show();
}
```

- Define the event handlers for the **InterstitialAd** object.

JavaScript

```
function adReadyHandler(sender) {
    // Your code goes here.
}

function errorOccurredHandler(sender, args) {
    // Your code goes here.
}

function completedHandler(sender) {
    // Your code goes here.
}

function cancelledHandler(sender) {
    // Your code goes here.
}
```

- Build and test your app to confirm it is showing test ads.

C++ (DirectX Interop)

This sample assumes you have created a C++ **DirectX and XAML App (Universal Windows)** project in Visual Studio and are targeting a specific CPU architecture.

- Open your project in Visual Studio.
- Add a reference to the Microsoft Advertising SDK in your project:

- a. From the **Solution Explorer** window, right click **References**, and select **Add Reference...**
 - b. In **Reference Manager**, expand **Universal Windows**, click **Extensions**, and then select the check box next to **Microsoft Advertising SDK for XAML** (Version 10.0).
 - c. In **Reference Manager**, click OK.
3. In an appropriate header file for your app (for example, `DirectXPage.xaml.h`), declare an **InterstitialAd** object and related event handler methods.

C++

```
Microsoft::Advertising::WinRT::UI::InterstitialAd^ m_interstitialAd;  
void OnAdReady(Object^ sender, Object^ args);  
void OnAdCompleted(Object^ sender, Object^ args);  
void OnAdCancelled(Object^ sender, Object^ args);  
void OnAdError(Object^ sender,  
Microsoft::Advertising::WinRT::UI::AdErrorEventArgs^ args);
```

4. In the same header file, declare several string fields that represent the application ID and ad unit ID for your interstitial ad. The following code example assigns the `myAppId` and `myAdUnitId` fields to the **test values** for interstitial ads.

ⓘ Note

Every **InterstitialAd** has a corresponding *ad unit* that is used by our services to serve ads to the control, and every ad unit consists of an *ad unit ID* and *application ID*. In these steps, you assign test ad unit ID and application ID values to your control. These test values can only be used in a test version of your app. Before you publish your app to the Store, you must **replace these test values with live values** from Partner Center.

C++

```
Platform::String^ myAppId = L"d25517cb-12d4-4699-8bdc-52040c712cab";  
Platform::String^ myAdUnitId = L"test";
```

5. In the `.cpp` file where you want to add code to show an interstitial ad, add the following namespace reference. The following examples assume you are adding the code to `DirectXPage.xaml.cpp` file in your app.

C++

```
using namespace Microsoft::Advertising::WinRT::UI;
```

6. In code that runs on startup (for example, in the constructor for the page), instantiate the **InterstitialAd** object and wire up event handlers for events of the object. In the following example, `InterstitialAdSamplesCpp` is the namespace for your project; change this name as necessary for your code.

C++

```
m_interstitialAd = ref new InterstitialAd();
m_interstitialAd->AdReady += ref new
Windows::Foundation::EventHandler<Platform::Object ^>
    (this, &InterstitialAdSamplesCpp::DirectXPage::OnAdReady);
m_interstitialAd->Completed += ref new
Windows::Foundation::EventHandler<Platform::Object ^>
    (this, &InterstitialAdSamplesCpp::DirectXPage::OnAdCompleted);
m_interstitialAd->Cancelled += ref new
Windows::Foundation::EventHandler<Platform::Object ^>
    (this, &InterstitialAdSamplesCpp::DirectXPage::OnAdCancelled);
m_interstitialAd->ErrorOccurred += ref new
Windows::Foundation::EventHandler<Microsoft::Advertising::WinRT::UI::Ad
ErrorEventArgs ^>
    (this, &InterstitialAdSamplesCpp::DirectXPage::OnAdError);
```

7. If you want to show an *interstitial video* ad: Approximately 30-60 seconds before you need the interstitial ad, use the `RequestAd` method to pre-fetch the ad. This allows enough time to request and prepare the ad before it should be shown. Be sure to specify **AdType::Video** for the ad type.

C++

```
m_interstitialAd->RequestAd(AdType::Video, myAppId, myAdUnitId);
```

If you want to show an *interstitial banner* ad: Approximately 5-8 seconds before you need the ad, use the `RequestAd` method to pre-fetch the ad. This allows enough time to request and prepare the ad before it should be shown. Be sure to specify **AdType::Display** for the ad type.

C++

```
m_interstitialAd->RequestAd(AdType::Display, myAppId, myAdUnitId);
```

- At the point in your code where you want to show the ad, confirm that the **InterstitialAd** is ready to be shown and then show it by using the [Show](#) method.

C++

```
if ((InterstitialAdState::Ready == m_interstitialAd->State))
{
    m_interstitialAd->Show();
}
```

- Define the event handlers for the **InterstitialAd** object.

C++

```
void DirectXPage::OnAdReady(Object^ sender, Object^ args)
{
    // Your code goes here.
}

void DirectXPage::OnAdCompleted(Object^ sender, Object^ args)
{
    // Your code goes here.
}

void DirectXPage::OnAdCancelled(Object^ sender, Object^ args)
{
    // Your code goes here.
}

void DirectXPage::OnAdError(Object^ sender,
Microsoft::Advertising::WinRT::UI::AdErrorEventArgs^ args)
{
    // Your code goes here.
}
```

- Build and test your app to confirm it is showing test ads.

Release your app with live ads

- Make sure your use of interstitial ads in your app follows our [guidelines for interstitial ads](#).
 - In Partner Center, go to the [In-app ads](#) page and [create an ad unit](#). For the ad unit type, choose **Video interstitial** or **Banner interstitial**, depending on what type of interstitial ad you are showing. Make note of both the ad unit ID and the application ID.
-

ⓘ Note

The application ID values for test ad units and live UWP ad units have different formats. Test application ID values are GUIDs. When you create a live UWP ad unit in Partner Center, the application ID value for the ad unit always matches the Store ID for your app (an example Store ID value looks like 9NBLGGH4R315).

3. You can optionally enable ad mediation for the **InterstitialAd** by configuring the settings in the [Mediation settings](#) section on the [In-app ads](#) page. Ad mediation enables you to maximize your ad revenue and app promotion capabilities by displaying ads from multiple ad networks, including ads from other paid ad networks such as Taboola and Smaato and ads for Microsoft app promotion campaigns.
4. In your code, replace the test ad unit values with the live values you generated in Partner Center.
5. [Submit your app](#) to the Store using Partner Center.
6. Review your [advertising performance reports](#) in Partner Center.

Manage ad units for multiple interstitial ad controls in your app

You can use multiple **InterstitialAd** controls in a single app. In this scenario, we recommend that you assign a different ad unit to each control. Using different ad units for each control enables you to separately [configure the mediation settings](#) and get discrete [reporting data](#) for each control. This also enables our services to better optimize the ads we serve to your app.

ⓘ Important

You can use each ad unit in only one app. If you use an ad unit in more than one app, ads will not be served for that ad unit.

Related topics

- [Guidelines for interstitial ads](#)

- [Interstitial ad sample code in C#](#)
- [Interstitial ad sample code in JavaScript](#)
- [Advertising samples on GitHub](#) ↗
- [Set up ad units for your app](#)

Interstitial ad sample code in C#

Article • 10/20/2022

Warning

As of June 1, 2020, the Microsoft Ad Monetization platform for Windows UWP apps will be shut down. [Learn more](#) .

This topic provides the complete sample code for a basic C# and XAML Universal Windows Platform (UWP) app that shows an interstitial video ad. For step-by-step instructions that show how to configure your project to use this code, see [Interstitial ads](#). For a complete sample project, see the [advertising samples on GitHub](#) .

Code example

This section shows the contents of the MainPage.xaml and MainPage.xaml.cs files in a basic app that shows an interstitial ad. To use these examples, copy the code into a Visual C# **Blank App (Universal Windows)** project in Visual Studio.

This sample app uses two buttons to request and then launch an interstitial ad. Replace the values of the `myAppId` and `myAdUnitId` fields with live values from Partner Center before submitting your app to the Store. For more information, see [Set up ad units in your app](#).

Note

To alter this example to show an interstitial banner ad instead of an interstitial video ad, pass the value `AdType.Display` to the first parameter of the `RequestAd` method instead of `AdType.Video`. For more information, see [Interstitial ads](#).

MainPage.xaml

XML

```
<Page
    x:Class="InterstitialAdSamplesCSharp.MainPage"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:local="using:InterstitialAdSamplesCSharp"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
```

```

xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
mc:Ignorable="d">
<Grid Background="{ThemeResource ApplicationPageBackgroundThemeBrush}">
    <Button Name="requestAdButton" Content="Request ad"
Margin="37,244,0,364" Click="requestAdButton_Click"/>
    <Button Name="showAdButton" Content="Show ad" Margin="37,309,0,299"
Click="showAdButton_Click"/>
</Grid>
</Page>

```

MainPage.xaml.cs

C#

```

using System;
using System.Collections.Generic;
using System.IO;
using Windows.UI.Xaml;
using Windows.UI.Xaml.Controls;
using Microsoft.Advertising.WinRT.UI;

namespace InterstitialAdSamplesCSharp
{
    public sealed partial class MainPage : Page
    {
        // Assign myAppId and myAdUnitId to test values. Replace these
        values with live values
        // from Dev Center before you submit your app to the Store.
        InterstitialAd myInterstitialAd = null;
        string myAppId = "d25517cb-12d4-4699-8bdc-52040c712cab";
        string myAdUnitId = "test";

        public MainPage()
        {
            this.InitializeComponent();

            myInterstitialAd = new InterstitialAd();
            myInterstitialAd.AdReady += MyInterstitialAd_AdReady;
            myInterstitialAd.ErrorOccurred +=
MyInterstitialAd_ErrorOccurred;
            myInterstitialAd.Completed += MyInterstitialAd_Completed;
            myInterstitialAd.Cancelled += MyInterstitialAd_Cancelled;
        }

        // This method requests an interstitial ad when the "Request ad"
        button is clicked. In a real app,
        // you should request the interstitial ad close to when you think it
        will be shown, but with
        // enough advance time to make the request and prepare the ad (say
        30 seconds to a few minutes).
        // To show an interstitial banner ad instead of an interstitial
        video ad, replace AdType.Video
    }
}

```

```

// with AdType.Display.
private void requestAdButton_Click(object sender, RoutedEventArgs e)
{
    myInterstitialAd.RequestAd(AdType.Video, myAppId, myAdUnitId);
}

// This method attempts to show the interstitial ad when the "Show
ad" button is clicked.
private void showAdButton_Click(object sender, RoutedEventArgs e)
{
    if (InterstitialAdState.Ready == myInterstitialAd.State)
    {
        myInterstitialAd.Show();
    }
}

void MyInterstitialAd_AdReady(object sender, object e)
{
    // Your code goes here.
}

void MyInterstitialAd_ErrorOccurred(object sender, AdErrorEventArgs
e)
{
    // Your code goes here.
}

void MyInterstitialAd_Completed(object sender, object e)
{
    // Your code goes here.
}

void MyInterstitialAd_Cancelled(object sender, object e)
{
    // Your code goes here.
}
    }
}

```

Related topics

- [Advertising samples on GitHub](#) 

Interstitial ad sample code in JavaScript

Article • 10/20/2022

Warning

As of June 1, 2020, the Microsoft Ad Monetization platform for Windows UWP apps will be shut down. [Learn more](#) .

This topic provides the complete sample code for a basic JavaScript and HTML Universal Windows Platform (UWP) app that shows an interstitial ad. For step-by-step instructions that show how to configure your project to use this code, see [Interstitial ads](#). For a complete sample project, see the [advertising samples on GitHub](#) .

Code example

This section shows the contents of the HTML and JavaScript files in a basic app that shows an interstitial ad. To use these examples, copy this code into a JavaScript **WinJS App (Universal Windows)** project in Visual Studio.

This sample app uses two buttons to request and then launch an interstitial ad. The `main.js` and `index.html` files generated by Visual Studio have been modified and are shown below. The `script.js` file shown below contains most of the code in the sample, and you should add this file to the `js` folder in your project.

Replace the values of the `applicationId` and `adUnitId` variables with live values from Partner Center before submitting your app to the Store. For more information, see [Set up ad units in your app](#).

Note

To alter this example to show an interstitial banner ad instead of an interstitial video ad, pass the value `InterstitialAdType.display` to the first parameter of the `RequestAd` method instead of `InterstitialAdType.video`. For more information, see [Interstitial ads](#).

index.html

HTML

```

<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>InterstitialAdSamplesJS</title>
  <link href="lib/winjs-4.0.1/css/ui-light.css" rel="stylesheet" />
  <script src="lib/winjs-4.0.1/js/base.js"></script>
  <script src="lib/winjs-4.0.1/js/ui.js"></script>
  <link href="css/default.css" rel="stylesheet" />
  <!-- JavaScript files containing code for this example -->
  <script src="js/main.js"></script>
  <script src="js/script.js"></script>
  <!-- Required reference for InterstitialAd -->
  <script src="//Microsoft.Advertising.JavaScript/ad.js"></script>
</head>
<body class="win-type-body">
  <button id="requestAdButton" onclick="requestAdButtonClick" class="win-button">Request ad</button>
  <button id="showAdButton" onclick="showAdButtonClick" class="win-button">Show ad</button>
  <div id="description" style="height:100%; overflow:auto"></div>
</body>
</html>

```

script.js

JavaScript

```

(function () {
  "use strict";

  // Assign applicationId and adUnitId to test values. Replace these
  // values with live values
  // from Dev Center before you submit your app to the Store.
  var interstitialAd = null;
  var applicationId = "d25517cb-12d4-4699-8bdc-52040c712cab";
  var adUnitId = "test";

  window.startInterstitial = function () {
    writeText("<br>Interstitial ads in JavaScript UWP apps");
    registerButtonEvents();

    // Initialize the InterstitialAd object and set up event handlers
    // for it.
    prepareInterstitial();

    writeText("Press the buttons to request and show an interstitial
    ad.");
  };

  var registerButtonEvents = function () {

```

```

        requestAdButton.addEventListener("click", requestAdButtonClick);
        showAdButton.addEventListener("click", showAdButtonClick);
    };

    // This example requests an interstitial ad when the "Request ad" button
    is clicked. In a real app,
    // you should request the interstitial ad close to when you think it
    will be shown, but with
    // enough advance time to make the request and prepare the ad (say 30
    seconds to a few minutes).
    // To show an interstitial banner ad instead of an interstitial video
    ad, replace InterstitialAdType.video
    // with InterstitialAdType.display.
    var requestAdButtonClick = function (evt) {
        if (interstitialAd) {

interstitialAd.requestAd(MicrosoftNSJS.Advertising.InterstitialAdType.video,
applicationId, adUnitId);
        }
    }

    // This example attempts to show the interstitial ad when the "Show ad"
    button is clicked.
    var showAdButtonClick = function (evt) {
        if (interstitialAd && interstitialAd.state !==
MicrosoftNSJS.Advertising.InterstitialAdState.showing) {
            showInterstitial();
        }
    }

    var restart = function () {
        if (interstitialAd) {
            interstitialAd.dispose();
        }
        interstitialAd = null;
        window.startInterstitial();
    };

    var clearText = function (msg) {
        description.innerHTML = "";
    };

    var writeText = function (msg) {
        description.innerHTML = description.innerHTML + msg + "<br>";
        description.scrollTop = description.scrollHeight;
    };

    var prepareInterstitial = function () {
        if (!interstitialAd) {
            interstitialAd = new MicrosoftNSJS.Advertising.InterstitialAd();
            interstitialAd.onErrorOccurred = errorOccurredHandler;
            interstitialAd.onAdReady = adReadyHandler;
            interstitialAd.onCancelled = cancelledHandler;
            interstitialAd.onCompleted = completedHandler;
        }
    }

```

```

    });

    var showInterstitial = function () {
        if (interstitialAd && interstitialAd.state ===
MicrosoftNSJS.Advertising.InterstitialAdState.ready) {
            interstitialAd.show();
        } else {
            // No ad is available to show. Allow user to try again anyway
            clearText();
            writeText("<br>Unable to show an ad. Check the error log. You
can try again.");
            restart();
        }
    };

    var errorOccurredHandler = function (sender, args) {
        console.log("error: " + args.errorMessage + " (" + args.errorCode +
")");
        if (!isPlaying) {
            clearText();
            writeText("<br>Unable to show an ad. Check the error log. You
can try again.");
            restart();
        }
    };

    var adReadyHandler = function (sender) {
        console.log("Ad ready");
    };

    var cancelledHandler = function (sender) {
        console.log("Ad cancelled");
        writeText("<br>You must watch the entire ad to continue. <b>Press
the button to watch the ad.</b>");
        interstitialAd.dispose();
        interstitialAd = null;
        prepareInterstitial();
    };

    var completedHandler = function (sender) {
        console.log("Ad complete");
        clearText();
        writeText("<br>Thanks for watching the ad! You can try again!");
        restart();
    };

    })());

```

main.js

```

(function () {
    "use strict";

    var app = WinJS.Application;
    var activation = Windows.ApplicationModel.Activation;
    var isFirstActivation = true;

    app.onactivated = function (args) {
        if (args.detail.kind === activation.ActivationKind.voiceCommand) {
            // TODO: Handle relevant ActivationKinds. For example, if your
            app can be started by voice commands,
            // this is a good place to decide whether to populate an input
            field or choose a different initial view.
        }
        else if (args.detail.kind === activation.ActivationKind.launch) {
            // A Launch activation happens when the user launches your app
            via the tile
            // or invokes a toast notification by clicking or tapping on the
            body.
            if (args.detail.arguments) {
                // TODO: If the app supports toasts, use this value from the
                toast payload to determine where in the app
                // to take the user in response to them invoking a toast
                notification.
            }
            else if (args.detail.previousExecutionState ===
activation.ApplicationExecutionState.terminated) {
                // TODO: This application had been suspended and was then
                terminated to reclaim memory.
                // To create a smooth user experience, restore application
                state here so that it looks like the app never stopped running.
                // Note: You may want to record the time when the app was
                last suspended and only restore state if they've returned after a short
                period.
            }
        }

        if (!args.detail.prelaunchActivated) {
            // TODO: If prelaunchActivated were true, it would mean the app
            was prelaunched in the background as an optimization.
            // In that case it would be suspended shortly thereafter.
            // Any long-running operations (like expensive network or disk
            I/O) or changes to user state which occur at launch
            // should be done here (to avoid doing them in the prelaunch
            case).
            // Alternatively, this work can be done in a resume or
            visibilitychanged handler.
        }

        if (isFirstActivation) {
            // TODO: The app was activated and had not been running. Do
            general startup initialization here.
            document.addEventListener("visibilitychange",
onVisibilityChanged);

```

```

        // Initialize interstitial ad support.
        startInterstitial();

        args.setPromise(WinJS.UI.processAll());
    }

    isFirstActivation = false;
};

function onVisibilityChanged(args) {
    if (!document.hidden) {
        // TODO: The app just became visible. This may be a good time to
        refresh the view.
    }
}

app.oncheckpoint = function (args) {
    // TODO: This application is about to be suspended. Save any state
    that needs to persist across suspensions here.
    // You might use the WinJS.Application.sessionState object, which is
    automatically saved and restored across suspension.
    // If you need to complete an asynchronous operation before your
    application is suspended, call args.setPromise().
};

app.start();

})();

```

Related topics

- [Advertising samples on GitHub](#) 

Native ads

Article • 01/04/2023

Warning

As of June 1, 2020, the Microsoft Ad Monetization platform for Windows UWP apps will be shut down. [Learn more](#) 

A native ad is a component-based ad format where each piece of the ad creative (such as the title, image, description, and call-to-action text) is delivered to your app as an individual element. You can integrate these elements into your app using your own fonts, colors, animations, and other UI components to stitch together an unobtrusive user experience that fits the look and feel of your app while also earning high yield from the ads.

For advertisers, native ads provide high-performing placements, because the ad experience is tightly integrated into the app and users therefore tend to interact more with these types of ads.

Note

Native ads are currently supported only for XAML-based UWP apps for Windows 10 and Windows 11. Support for UWP apps written using HTML and JavaScript is planned for a future release of the Microsoft Advertising SDK.

Prerequisites

- Install the [Microsoft Advertising SDK](#)  with Visual Studio 2015 or a later release of Visual Studio. For installation instructions, see [this article](#).

Integrate a native ad into your app

Follow these instructions to integrate a native ad into your app and confirm that your native ad implementation shows a test ad.

1. In Visual Studio, open your project or create a new project.

Note

If you're using an existing project, open the Package.appxmanifest file in your project and ensure that the **Internet (Client)** capability is selected. Your app needs this capability to receive test ads and live ads.

2. If your project targets **Any CPU**, update your project to use an architecture-specific build output (for example, **x86**). If your project targets **Any CPU**, you will not be able to successfully add a reference to the Microsoft Advertising SDK in the following steps. For more information, see [Reference errors caused by targeting Any CPU in your project](#).
3. Add a reference to the Microsoft Advertising SDK in your project:
 - a. From the **Solution Explorer** window, right click **References**, and select **Add Reference...**
 - b. In **Reference Manager**, expand **Universal Windows**, click **Extensions**, and then select the check box next to **Microsoft Advertising SDK for XAML** (Version 10.0).
 - c. In **Reference Manager**, click OK.
4. In the appropriate code file in your app (for example, in MainPage.xaml.cs or a code file for some other page), add the following namespace references.

```
C#  
  
using Microsoft.Advertising.WinRT.UI;  
using Windows.UI.Xaml.Media.Imaging;
```

5. In an appropriate location in your app (for example, in `MainPage` or some other page), declare a `NativeAdsManagerV2` object and several string fields that represent the application ID and ad unit ID for your native ad. The following code example assigns the `myAppId` and `myAdUnitId` fields to the [test values](#) for native ads.

ⓘ Note

Every **NativeAdsManagerV2** has a corresponding *ad unit* that is used by our services to serve ads to the native ad control, and every ad unit consists of an *ad unit ID* and *application ID*. In these steps, you assign test ad unit ID and application ID values to your control. These test values can only be used in a test version of your app. Before you publish your app to the Store, you must **replace these test values with live values** from Partner Center.

C#

```
NativeAdsManagerV2 myNativeAdsManager = null;  
string myAppId = "d25517cb-12d4-4699-8bdc-52040c712cab";  
string myAdUnitId = "test";
```

6. In code that runs on startup (for example, in the constructor for the page), instantiate the **NativeAdsManagerV2** object and wire up event handlers for the **AdReady** and **ErrorOccurred** events of the object.

C#

```
myNativeAdsManager = new NativeAdsManagerV2(myAppId, myAdUnitId);  
myNativeAdsManager.AdReady += MyNativeAd_AdReady;  
myNativeAdsManager.ErrorOccurred += MyNativeAdsManager_ErrorOccurred;
```

7. When you're ready to show a native ad, call the **RequestAd** method to fetch an ad.

C#

```
myNativeAdsManager.RequestAd();
```

8. When a native ad is ready for your app, your **AdReady** event handler is called, and a **NativeAdV2** object that represents the native ad is passed to the *e* parameter. Use the **NativeAdV2** properties to get each element of the native ad and display these elements on your page. Be sure to also call the **RegisterAdContainer** method to register the UI element that acts as a container for the native ad; this is required to properly track ad impressions and clicks.

ⓘ Note

Some elements of the native ad are required and must always be shown in your app. For more information, see our [guidelines for native ads](#).

For example, assume that your app contains a `MainPage` (or some other page) with the following **StackPanel**. This **StackPanel** contains a series of controls that display different elements of a native ad, including the title, description, images, *sponsored* by text, and a button that will show the *call to action* text.

XML

```
<StackPanel x:Name="NativeAdContainer" Background="#555555"  
Width="Auto" Height="Auto"
```

```

        Orientation="Vertical">
            <Image x:Name="AdIconImage" HorizontalAlignment="Left"
VerticalAlignment="Center"
                Margin="20,20,20,20"/>
            <TextBlock x:Name="TitleTextBlock" HorizontalAlignment="Left"
VerticalAlignment="Center"
                Text="The ad title will go here" FontSize="24"
Foreground="White" Margin="20,0,0,10"/>
            <TextBlock x:Name="DescriptionTextBlock" HorizontalAlignment="Left"
VerticalAlignment="Center"
                Foreground="White" TextWrapping="Wrap" Text="The ad
description will go here"
                Margin="20,0,0,0" Visibility="Collapsed"/>
            <Image x:Name="MainImageImage" HorizontalAlignment="Left"
VerticalAlignment="Center" Margin="20,20,20,20"
Visibility="Collapsed"/>
            <Button x:Name="CallToActionButton" Background="Gray"
Foreground="White"
                HorizontalAlignment="Left" VerticalAlignment="Center"
Width="Auto" Height="Auto"
                Content="The call to action text will go here"
Margin="20,20,20,20"
                Visibility="Collapsed"/>
            <StackPanel x:Name="SponsoredByStackPanel" Orientation="Horizontal"
Margin="20,20,20,20">
                <TextBlock x:Name="SponsoredByTextBlock" Text="The ad sponsored
by text will go here"
                    FontSize="24" Foreground="White" Margin="20,0,0,0"
HorizontalAlignment="Left"
                    VerticalAlignment="Center" Visibility="Collapsed"/>
                <Image x:Name="IconImageImage" Margin="40,20,20,20"
HorizontalAlignment="Left"
                    VerticalAlignment="Center" Visibility="Collapsed"/>
            </StackPanel>
        </StackPanel>
    </StackPanel>

```

The following code example demonstrates an **AdReady** event handler that displays each element of the native ad in the controls in the **StackPanel** and then calls the **RegisterAdContainer** method to register the **StackPanel**. This code assumes that it is run from the code-behind file for the page that contains the **StackPanel**.

C#

```

void MyNativeAd_AdReady(object sender, NativeAdReadyEventArgs e)
{
    NativeAdV2 nativeAd = e.NativeAd;

    // Show the ad icon.
    if (nativeAd.AdIcon != null)
    {
        AdIconImage.Source = nativeAd.AdIcon.Source;
    }
}

```

```

        // Adjust the Image control to the height and width of the
        // provided ad icon.
        AdIconImage.Height = nativeAd.AdIcon.Height;
        AdIconImage.Width = nativeAd.AdIcon.Width;
    }

    // Show the ad title.
    TitleTextBlock.Text = nativeAd.Title;

    // Show the ad description.
    if (!string.IsNullOrEmpty(nativeAd.Description))
    {
        DescriptionTextBlock.Text = nativeAd.Description;
        DescriptionTextBlock.Visibility = Visibility.Visible;
    }

    // Display the first main image for the ad. Note that the service
    // might provide multiple main images.
    if (nativeAd.MainImages.Count > 0)
    {
        NativeImage mainImage = nativeAd.MainImages[0];
        BitmapImage bitmapImage = new BitmapImage();
        bitmapImage.UriSource = new Uri(mainImage.Url);
        MainImageImage.Source = bitmapImage;

        // Adjust the Image control to the height and width of the
        // main image.
        MainImageImage.Height = mainImage.Height;
        MainImageImage.Width = mainImage.Width;
        MainImageImage.Visibility = Visibility.Visible;
    }

    // Add the call to action string to the button.
    if (!string.IsNullOrEmpty(nativeAd.CallToActionText))
    {
        CallToActionButton.Content = nativeAd.CallToActionText;
        CallToActionButton.Visibility = Visibility.Visible;
    }

    // Show the ad sponsored by value.
    if (!string.IsNullOrEmpty(nativeAd.SponsoredBy))
    {
        SponsoredByTextBlock.Text = nativeAd.SponsoredBy;
        SponsoredByTextBlock.Visibility = Visibility.Visible;
    }

    // Show the icon image for the ad.
    if (nativeAd.IconImage != null)
    {
        BitmapImage bitmapImage = new BitmapImage();
        bitmapImage.UriSource = new Uri(nativeAd.IconImage.Url);
        IconImageImage.Source = bitmapImage;

        // Adjust the Image control to the height and width of the
        // icon image.
    }

```

```

        IconImageImage.Height = nativeAd.IconImage.Height;
        IconImageImage.Width = nativeAd.IconImage.Width;
        IconImageImage.Visibility = Visibility.Visible;
    }

    // Register the container of the controls that display
    // the native ad elements for clicks/impressions.
    nativeAd.RegisterAdContainer(NativeAdContainer);
}

```

9. Define an event handler for the **ErrorOccurred** event to handle errors related to the native ad. The following example writes error information to the Visual Studio **Output** window during testing.

```

C#

private void MyNativeAdsManager_ErrorOccurred(object sender,
NativeAdErrorEventArgs e)
{
    System.Diagnostics.Debug.WriteLine("NativeAd error " +
e.ErrorMessage +
    " ErrorCode: " + e.ErrorCode.ToString());
}

```

10. Compile and run the app to see it with a test ad.

Release your app with live ads

After you confirm that your native ad implementation successfully shows a test ad, follow these instructions to configure your app to show real ads and submit your updated app to the Store.

1. Make sure that your native ad implementation follows our [guidelines for native ads](#).
2. In Partner Center, go to the [In-app ads](#) page and [create an ad unit](#). For the ad unit type, specify **Native**. Make note of both the ad unit ID and the application ID.

ⓘ Note

The application ID values for test ad units and live UWP ad units have different formats. Test application ID values are GUIDs. When you create a live UWP ad unit in Partner Center, the application ID value for the ad unit always matches the Store ID for your app (an example Store ID value looks like 9NBLGGH4R315).

3. You can optionally enable ad mediation for the native ad by configuring the settings in the [Mediation settings](#) section on the [In-app ads](#) page. Ad mediation enables you to maximize your ad revenue and app promotion capabilities by displaying ads from multiple ad networks.
4. In your code, replace the test ad unit values (that is, the *applicationId* and *adUnitId* parameters of the [NativeAdsManagerV2](#) constructor) with the live values you generated in Partner Center.
5. [Submit your app](#) to the Store using Partner Center.
6. Review your [advertising performance reports](#) in Partner Center.

Manage ad units for multiple native ads in your app

You can use multiple native ad placements in a single app. In this scenario, we recommend that you assign a different ad unit to each native ad placements. Using different ad units for native ad enables you to separately [configure the mediation settings](#) and get discrete [reporting data](#) for each control. This also enables our services to better optimize the ads we serve to your app.

Important

You can use each ad unit in only one app. If you use an ad unit in more than one app, ads will not be served for that ad unit.

Related topics

- [Guidelines for native ads](#)
- [In-app ads](#)
- [Set up ad units for your app](#)

Show ads in video content

Article • 03/16/2023

⚠ Warning

As of June 1, 2020, the Microsoft Ad Monetization platform for Windows UWP apps will be shut down. [Learn more](#) ↗

This walkthrough shows how to use the **AdScheduler** class to show ads in video content in a Universal Windows Platform (UWP) app that was written using JavaScript with HTML.

📌 Note

This feature is currently supported only for UWP apps that are written using JavaScript with HTML.

AdScheduler works with both progressive and streaming media, and uses IAB standard Video Ad Serving Template (VAST) 2.0/3.0 and VMAP payload formats. By using standards, **AdScheduler** is agnostic to the ad service with which it interacts.

Advertising for video content differs based upon whether the program is under ten minutes (short form), or over ten minutes (long form). Although the latter is more complicated to set up on the service, there is actually no difference in how one writes the client side code. If the **AdScheduler** receives a VAST payload with a single ad instead of a manifest, it is treated as if the manifest called for a single pre-roll ad (one break at time 00:00).

Prerequisites

- Install the [Microsoft Advertising SDK](#) ↗ with Visual Studio 2015 or a later release.
- Your project must use the [MediaPlayer](#) ↗ control to serve the video content in which the ads will be scheduled. This control is available in the [TVHelpers](#) ↗ collection of libraries available from Microsoft on GitHub.

The following example shows how to declare a [MediaPlayer](#) ↗ in HTML markup. Typically, this markup belongs in the `<body>` section in the index.html file (or another html file as appropriate for your project).

HTML

```
<div id="MediaPlayerDiv" data-win-control="TVJS.MediaPlayer">
  <video src="URL to your content">
</video>
</div>
```

The following example shows how to establish a [MediaPlayer](#) in JavaScript code.

JavaScript

```
var mediaPlayerDiv = document.createElement("div");
document.body.appendChild(mediaPlayerDiv);

var videoElement = document.createElement("video");
videoElement.src = "<URL to your content>";
mediaPlayerDiv.appendChild(videoElement);

var mediaPlayer = new TVJS.MediaPlayer(mediaPlayerDiv);
```

How to use the AdScheduler class in your code

1. In Visual Studio, open your project or create a new project.
2. If your project targets **Any CPU**, update your project to use an architecture-specific build output (for example, **x86**). If your project targets **Any CPU**, you will not be able to successfully add a reference to the Microsoft advertising library in the following steps. For more information, see [Reference errors caused by targeting Any CPU in your project](#).
3. Add a reference to the **Microsoft Advertising SDK for JavaScript** library to your project.
 - a. From the **Solution Explorer** window, right click **References**, and select **Add Reference...**
 - b. In **Reference Manager**, expand **Universal Windows**, click **Extensions**, and then select the check box next to **Microsoft Advertising SDK for JavaScript** (Version 10.0).
 - c. In **Reference Manager**, click **OK**.
4. Add the AdScheduler.js file to your project:
 - a. In Visual Studio, click **Project** and **Manage NuGet Packages**.
 - b. In the search box, type **Microsoft.StoreServices.VideoAdScheduler** and install the Microsoft.StoreServices.VideoAdScheduler package. The AdScheduler.js file is added to the **../js** subdirectory in your project.

5. Open the `index.html` file (or other html file as appropriate for your project). In the `<head>` section, after the project's JavaScript references of `default.css` and `main.js`, add the reference to `ad.js` and `adscheduler.js`.

HTML

```
<script src="//Microsoft.Advertising.Javascript/ad.js"></script>
<script src="/js/adscheduler.js"></script>
```

ⓘ Note

This line must be placed in the `<head>` section after the include of `main.js`; otherwise, you will encounter an error when you build your project.

6. In the `main.js` file in your project, add code that creates a new **AdScheduler** object. Pass in the **MediaPlayer** that hosts your video content. The code must be placed so that it runs after `WinJS.UI.processAll`.

JavaScript

```
var myMediaPlayer = document.getElementById("MediaPlayerDiv");
var myAdScheduler = new
MicrosoftNSJS.Advertising.AdScheduler(myMediaPlayer);
```

7. Use the **requestSchedule** or **requestScheduleByUrl** methods of the **AdScheduler** object to request an ad schedule from the server and insert it into the **MediaPlayer** timeline, and then play the video media.

- If you are a Microsoft partner who has received permission to request an ad schedule from the Microsoft ad server, use **requestSchedule** and specify the application ID and ad unit ID that were provided to you by your Microsoft representative.

This method takes the form of a [Promise](#), which is an asynchronous construct where two function pointers are passed: a pointer for the **onComplete** function to call when the promise completes successfully and a pointer for the **onError** function to call if an error is encountered. In the **onComplete** function, start playback of your video content. The ad will start playing at the scheduled time. In your **onError** function, handle the error and then start playback of your video. Your video content will play without an ad. The argument of the **onError** function is an object that contains the following members.

JavaScript

```
myAdScheduler.requestSchedule("your application ID", "your ad unit ID").then(  
    function promiseSuccessHandler(schedule) {  
        // Success: play the video content with the scheduled ads.  
        myMediaPlayer.tvControl.play();  
    },  
    function promiseErrorHandler(err) {  
        // Error: play the video content without the ads.  
        myMediaPlayer.tvControl.play();  
    });
```

- To request an ad schedule from a non-Microsoft ad server, use **requestScheduleByUrl**, and pass in the server URL. This method also takes the form of a **Promise** that accepts pointers for the **onComplete** and **onError** functions. The ad payload that is returned from the server must conform to the Video Ad Serving Template (VAST) or Video Multiple Ad Playlist (VMAP) payload formats.

JavaScript

```
myAdScheduler.requestScheduleByUrl("your URL").then(  
    function promiseSuccessHandler(schedule) {  
        // Success: play the video content with the scheduled ads.  
        myMediaPlayer.winControl.play();  
    },  
    function promiseErrorHandler(evt) {  
        // Error: play the video content without the ads.  
        myMediaPlayer.winControl.play();  
    });
```

ⓘ Note

You should wait for **requestSchedule** or **requestScheduleByUrl** to return before starting to play the primary video content in the **MediaPlayer**. Starting to play media before **requestSchedule** returns (in the case of a pre-roll advertisement) will result in the pre-roll interrupting the primary video content. You must call **play** even if the function fails, because the **AdScheduler** will tell the **MediaPlayer** to skip the ad(s) and move straight to the content. You may have a different business requirement, such as inserting a built-in ad if an ad can't be successfully fetched remotely.

8. During playback, you can handle additional events that let your app track progress and/or errors which may occur after the initial ad matching process. The following

code shows some of these events, including **onPodStart**, **onPodEnd**, **onPodCountdown**, **onAdProgress**, **onAllComplete**, and **onErrorOccurred**.

JavaScript

```
// Raised when an ad pod starts. Make the countdown timer visible.
myAdScheduler.onPodStart = function (sender, data) {
    myCounterDiv.style.visibility = "visible";
}

// Raised when an ad pod ends. Hide the countdown timer.
myAdScheduler.onPodEnd = function (sender, data) {
    myCounterDiv.style.visibility = "hidden";
}

// Raised when an ad is playing and indicates how many seconds remain
// in the current pod of ads. This is useful when the app wants to show
// a visual countdown until the video content resumes.
myAdScheduler.onPodCountdown = function (sender, data) {
    myCounterText = "Content resumes in: " +
        Math.ceil(data.remainingPodTime) + " seconds.";
}

// Raised during each quartile of progress through the ad clip.
myAdScheduler.onAdProgress = function (sender, data) {
}

// Raised when the ads and content are complete.
myAdScheduler.onAllComplete = function (sender) {
}

// Raised when an error occurs during playback after successfully
// scheduling.
myAdScheduler.onErrorOccurred = function (sender, data) {
}
```

AdScheduler members

This section provides some details about the members of the **AdScheduler** object. For more information about these members, see the comments and definitions in the **AdScheduler.js** file in your project.

requestSchedule

This method requests an ad schedule from the Microsoft ad server and inserts it into the timeline of the **MediaPlayer** that was passed to the **AdScheduler** constructor.

The optional third parameter (*adTags*) is a JSON collection of name/value pairs that can be used for apps that have advanced targeting. For example, an app that plays a variety of auto-related videos might supplement the ad unit ID with the make and model of the car being shown. This parameter is intended to be used only by partners who receive approval from Microsoft to use ad tags.

The following items should be noted when referring to *adTags*:

- This parameter is a very rarely used option. The publisher must work closely with Microsoft before using adTags.
- Both the names and the values must be predetermined on the ad service. Ad tags are not open ended search terms or keywords.
- The maximum supported number of tags is 10.
- Tag names are restricted to 16 characters.
- Tag values have a maximum of 128 characters.

requestScheduleByUri

This method requests an ad schedule from the non-Microsoft ad server specified in the URI and inserts it into the timeline of the **MediaPlayer** that was passed to the **AdScheduler** constructor. The ad payload that is returned by the ad server must conform to the Video Ad Serving Template (VAST) or Video Multiple Ad Playlist (VMAP) payload formats.

mediaTimeout

This property gets or sets the number of milliseconds that the media must be playable. A value of 0 informs the system to never timeout. The default is 30000 ms (30 seconds).

playSkippedMedia

This property gets or sets a **Boolean** value that indicates whether scheduled media will play if user skips ahead to a point past a scheduled start time.

The ad client and media player will enforce rules in terms what happens to advertisements during fast forwarding and rewinding of the primary video content. In most cases, app developers do not allow advertisements to be entirely skipped over but do want to provide a reasonable experience for the user. The following two options fall within the needs of most developers:

1. Allow end-users to skip over advertisement pods at will.

2. Allow users to skip over advertisement pods but play the most recent pod when playback resumes.

The **playSkippedMedia** property has the following conditions:

- Advertisements cannot be skipped or fast forwarded once the advertisement begins.
- All advertisements in an advertisement pod will play once the pod has started.
- Once played, an advertisement will not play again during the primary content (movie, episode, etc.); advertisement markers will be marked as played or removed.
- Pre-rollout advertisements cannot be skipped.

When resuming content that contains advertising, set **playSkippedMedia** to **false** to skip pre-rolls and prevent the most recent ad break from playing. Then, after the content starts, set **playSkippedMedia** to **true** to ensure that users cannot fast-forward through subsequent ads.

ⓘ Note

A pod is a group of ads that play in a sequence, such as a group of ads that play during a commercial break. For more details, see the IAB Digital Video Ad Serving Template (VAST) specification.

requestTimeout

This property gets or sets the number of milliseconds to wait for an ad request response before timing out. A value of 0 informs the system to never timeout. The default is 30000 ms (30 seconds).

schedule

This property gets the schedule data that was retrieved from the ad server. This object includes the full hierarchy of data that corresponds to the structure of the Video Ad Serving Template (VAST) or Video Multiple Ad Playlist (VMAP) payload.

onAdProgress

This event is raised when ad playback reaches quartile checkpoints. The second parameter of the event handler (*eventInfo*) is a JSON object with the following members:

- **progress**: The ad playback status (one of the **MediaProgress** enum values defined in `AdScheduler.js`).
- **clip**: The video clip that is being played. This object is not intended to be used in your code.
- **adPackage**: An object that represents the part of the ad payload that corresponds to the ad that is playing. This object is not intended to be used in your code.

onAllComplete

This event is raised when the main content reaches the end and any scheduled post-roll ads are also ended.

onErrorOccurred

This event is raised when the **AdScheduler** encounters an error. For more information about the error code values, see [ErrorCode](#).

onPodCountdown

This event is raised when an ad is playing and indicates how much time remains in the current pod. The second parameter of the event handler (*eventData*) is a JSON object with the following members:

- **remainingAdTime**: The number of seconds left for the current ad.
- **remainingPodTime**: The number of seconds left for the current pod.

ⓘ Note

A pod is a group of ads that play in a sequence, such as a group of ads that play during a commercial break. For more details, see the IAB Digital Video Ad Serving Template (VAST) specification.

onPodEnd

This event is raised when an ad pod ends. The second parameter of the event handler (*eventData*) is a JSON object with the following members:

- **startTime**: The pod's start time, in seconds.
- **pod**: An object that represents the pod. This object is not intended to be used in your code.

onPodStart

This event is raised when an ad pod starts. The second parameter of the event handler (*eventData*) is a JSON object with the following members:

- **startTime**: The pod's start time, in seconds.
- **pod**: An object that represents the pod. This object is not intended to be used in your code.

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Set up ad units in your app

Article • 12/03/2024

Warning

As of June 1, 2020, the Microsoft Ad Monetization platform for Windows UWP apps will be shut down. [Learn more](#) 

Every ad control in your Universal Windows Platform (UWP) app has a corresponding *ad unit* that is used by our services to serve ads to the control. Each ad unit consists of an *ad unit ID* and *application ID* that you must assign to code in your app.

We provide [test ad unit values](#) that you can use during testing to confirm that your app shows test ads. These test values can only be used in a test version of your app. If you try to use test values in your app after you publish it, your live app not receive ads.

After you finish testing your UWP app and you are ready to submit it to Partner Center, you must [create a live ad unit](#) from the [In-app ads](#) page in Partner Center and update your app code to use the application ID and ad unit ID values for this ad unit.

For more information about assigning the application ID and ad unit ID values in your app's code, see the following articles:

- [AdControl in XAML and .NET](#)
- [AdControl in HTML 5 and JavaScript](#)
- [Interstitial ads](#)
- [Native ads](#)

Test ad units

While you are developing your app, use the test application ID and ad unit ID values from this section to see how your app renders ads during testing.

Banner ads (using the AdControl class)

- Ad unit ID: `test`
- Application ID: `00001111-aaaa-2222-bbbb-3333cccc4444`

Important

For an **AdControl**, the size of a live ad is defined by the **Width** and **Height** properties. For best results, make sure that the **Width** and **Height** properties in your code are one of the [supported ad sizes for banner ads](#). The **Width** and **Height** properties will not change based on the size of a live ad.

Interstitial ads and native ads

- Ad unit ID: `test`
- Application ID: `11112222-bbbb-3333-cccc-4444dddd5555`

Live ad units

To get a live ad unit from Partner Center and use it in your app:

1. [Create an ad unit](#) on the **In-app ads** page in Partner Center. Be sure to specify the correct type of ad unit for the ad control you are using in your app.

ⓘ Note

You can optionally enable ad mediation for your ad unit by configuring the settings in the [Mediation settings](#) section. Ad mediation enables you to maximize your ad revenue and app promotion capabilities by displaying ads from multiple ad networks, including ads from other paid ad networks and ads for Microsoft app promotion campaigns. By default, we automatically choose the ad mediation settings for your app using machine-learning algorithms to help you maximize your ad revenue across the markets your app supports, but you can optionally manually configure your mediation settings.

2. After you create the new ad unit, retrieve the **Application ID** and **Ad unit ID** for the ad unit in the table of available ad units in the **Monetize > In-app ads** page.

ⓘ Note

The application ID values for test ad units and live UWP ad units have different formats. Test application ID values are GUIDs. When you create a live UWP ad unit in Partner Center, the application ID value for the ad unit always matches the Store ID for your app (an example Store ID value looks like 9NBLGGH4R315).

3. Assign the application ID and ad unit ID values in your app's code. For more information, see the following articles:

- [AdControl in XAML and .NET](#)
- [AdControl in HTML 5 and JavaScript](#)
- [Interstitial ads](#)
- [Native ads](#)

Manage ad units for multiple ad controls in your app

You can use multiple banner, interstitial, and native ad controls in a single app. In this scenario, we recommend that you assign a different ad unit to each control. Using different ad units for each control enables you to separately [configure the mediation settings](#) and get discrete [reporting data](#) for each control. This also enables our services to better optimize the ads we serve to your app.

Important

You can use each ad unit in only one app. If you use an ad unit in more than one app, ads will not be served for that ad unit.

Related topics

- [AdControl in XAML and .NET](#)
- [AdControl in HTML 5 and JavaScript](#)
- [Interstitial ads](#)
- [Native ads](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Microsoft ad mediation service

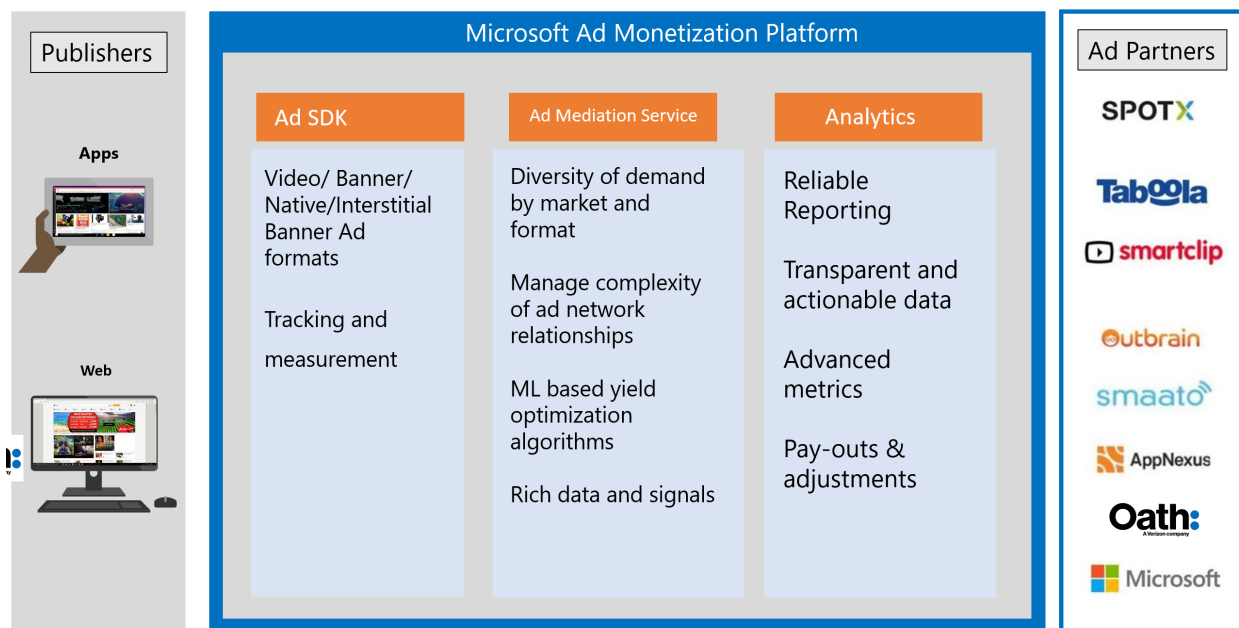
Article • 03/16/2023

⚠ Warning

As of June 1, 2020, the Microsoft Ad Monetization platform for Windows UWP apps will be shut down. [Learn more](#) ↗

When you use the [Microsoft Advertising SDK](#) to display ads in your apps, you can optionally use the Microsoft ad mediation service to maximize your ad revenue. This article provides an overview of the ad mediation service and its goals.

The ad mediation service is a part of the [Microsoft Ad Monetization platform](#). The platform is composed of the following parts.



The ad mediation service aims to maximize ad revenue for your apps by building on these capabilities.

Diversity of demand by market and format

The ad mediation service integrates with a variety of ad networks across the different ad formats (banner, interstitial banner, interstitial video, and native). The ad mediation service works with each ad network to ensure that they can serve ads with the highest potential for engaging the end user. We will continue to add different ad partners to expand the variety and quality of demand.

Manage complexity of ad network relationships

The ad mediation service integrates with a wide variety of ad networks so that you don't need to do this work. After you use the Microsoft Advertising SDK to display ads in your app, you can modify your ad mediation settings [in Partner Center](#) to display ads from multiple ad networks. You benefit from getting ads from new ad networks without having to make any changes to your code.

We manage the end-to-end relationship with the ad networks on your behalf. Everything from ad network integration to serving ads, reporting and payouts are taken care of by us with no additional effort from you.

Machine learning-based yield optimization algorithms

The ad mediation service works to generate the highest yield for developers. To do so, it employs machine learning-based yield optimization algorithms. For every ad call, the algorithms determine the best *waterfall* order in which the ad networks need to be called to maximize revenue for you. This is done by considering many factors, including:

- The user making the request.
- The application the request is made for.
- Past performance of the ad network.
- The ad format and market where the request is coming from.
- The time of the ad call.
- The category of the app content.
- The user segment.
- The user's location.

New ad networks are automatically included and evaluated for performance through a learning budget. Within a short period of time, they find their place in the waterfall. This makes the ad networks more competitive and helps the developer make the most of monetizing through apps.

We highly recommend using our [recommended mediation settings](#) to maximize revenue made from ads in your apps. This allows for our algorithms to enable the best yield for your app. However, you also have the freedom to choose your own mediation settings in Partner Center to have more control over the ad networks that serve ads and the order in which they do.

Rich data and signals

The ad mediation service works with various ad networks to improve user targeting to show more relevant ads for each user. This is done by sending richer signals about the user and the app to the ad network, while keeping privacy requirements in mind.

Related topics

- [Microsoft Advertising SDK](#) 
- [Mediation settings](#)
- [Advertising performance report](#)

Best practices for ads in apps

Article • 10/20/2022

Warning

As of June 1, 2020, the Microsoft Ad Monetization platform for Windows UWP apps will be shut down. [Learn more](#) 

The articles in this section provide user experience guidelines and error handling guidance for banner ads, interstitial ads, and native ads in your UWP apps.

In this section

Topic	Description
UI and user experience guidelines	Provides guidelines for providing great user experiences with banner ads and interstitial ads in your apps.
Optimize the viewability of your ad units	Provides guidelines for optimizing the viewable impressions for your ad units.
Handle ad errors	Learn how to handle errors for banner ads, interstitial ads, and native ads in your apps.